The end product is the entire infrastructure for an Online Newspaper. There will be a webpage for "supporters". Which will be like the people that work for the Newspaper. One for the main page of the Newspaper, where any people will be able to see the news. And one for the "Editors", which will be the people making the decisions for what comes out eventually in the newspaper, and have access to the infrastructure of the Newspaper as a whole.

The entire Infrastructure must be built in a modular system with a default directive in mind, posting the incoming article. There must be a panel where every step of the lifetime of the article can be shut off. If they are all on, the entire process will be necessary for the News article to be posted. If they are all off, all incoming work must be published.

All original systems like the priority badges and priority scales described below must be modular as well. In a way that new priority conditions, and badges can be added without needing to enter the source code directly.

The expected result will define 3 types of visitor:

> **None**: a person that simply accepts the cookies, and only views the news.
>
> **Editor**: A person with access to the infrastructure, (Admin side) of the Newspaper and helps make decisions for what eventually gets published.
>
> **Reporter**: A person that can make an account to see the news and receive Notifications, submit feedback and comments, or Not make it. May be authorized to submit news. Will receive notifications of news projects and submit what may be required. Writing, video, pictures of interviews, research etc.

The Editors and Reporters will have badges that will control the type of access and permissions that they have to their respective pages:

Editors:

Registered -> Basic Badge only visible to Admin. A user completed their registration and is waiting for approval. Approval may require an in-person ID check. For approval to be set, the admin must add to their profile the corresponding badges. Can only view their account info and edit it.

Deactivated -> Has no permissions, PII is saved on a blacklist database, during a specific amount of time accepted in the TOS. After which the account is terminated.

Admin -> The Mapache badge, that’s only for developers and can have access to everything. Can Schedule Editorial Meets. Can Approve Registrants. Can grant and remove badges. Can create badges and add priority scales.

Agent -> Can edit the priority of the articles and view all info in them. Has access to the priority queue. Can allocate resources to the articles and send them to the development stage. Receives Notifications when an Editorial Meet is scheduled.

Writer -> Can view all info in the articles from their queue. Has access to the archive and the approved articles. Can allocate resources to the articles and send them back to the development stage. Can edit the articles, include and remove information.

Director -> Has access to the finalized article queue. Can access the Composing Bay. Can select the final view of the article and schedule when they'll be published. Can Schedule Editorial Meets. Can Approve Registrants. Can grant and remove badges. Can approve exclusions from the weekly page view.

Coordinator -> Support role for the Director. Has all the same permission, except the one to publish the article.

Mod -> Can Schedule Editorial Meets. Can Approve Registrants. Can grant and remove badges

Designer -> Can access the composing bay. Has access to the finalized article queue with a weekly view. Can request to exclude articles from the weekly report with authorization of the Director or a Coordinator.

Reporters:

Viewer -> Basic Badge only visible to Admin. A valid user that hasn't completed their ID check. Has access to the Newspaper page. receive Notifications, submit feedback and comments,

Field -> A valid User that has completed their ID check. Has the same access as a Viewer. Can submit News requests. Receives information on their news updates. Their name appears on the article's Credit section. Can receive emails requesting more information. Interview/details/contact.

Deactivated -> Has no permissions, PII is saved on a blacklist database, during a specific amount of time accepted in the TOS. After which the account is terminated.

Restricted -> Same permissions as a Viewer. Won't be able to do their ID Check to become Field.

Photographer -> Same permissions as a Field. Can be allocated as a resource for an article requiring photographs. May receive notifications asking to get pictures for said article.

Videographer -> Same permissions as a Field. Can be allocated as a resource for an article requiring videos. May receive notifications asking to get video for said article.

Redacter -> Same permissions as a Field. Can be allocated as a resource for an article requiring redaction. May receive notifications asking to get a written piece for said article.

Interviewer -> Same permissions as a Field. Can be allocated as a resource for an article requiring an interview. May receive notifications asking to get an interview for said article.

Inspector -> Same permissions as a Field. Can be allocated as a resource for an article requiring attending an event and submitting any possible details about it. May receive notifications asking to attend an event to get information for said article.

This Users Will take part in the lifecycle of an article from submission to publication. Such lifecycle is described below:

Stage 1: Content Ingestion & Automated Screening

- **Global Contributor Submissions:** Journalists and correspondents from around the world submit reporting, pitches, and raw news materials into the platform.
- **Intake & Filtering:** Inbound submissions pass through an automated screening system (powered by AI) that performs an initial quality check to filter out non-viable proposals.
- **Direct Archiving & Bypassing:** Editorial staff can bypass the automated AI filter when necessary to route materials directly into the primary archive, which can later feed into the prioritization queue.

Stage 2: Editorial Evaluation & Priority Scoring

- **Thematic Alignment:** An assigned editor reviews the filtered submissions against the publication's current editorial line and weekly thematic focus (e.g., prioritizing environmental and natural disaster stories over localized crime).
- **Value & Impact Scoring:** Each qualifying story proposal is evaluated on a priority score based on public interest, scope of impact, and urgency to determine publication order, and the newspaper's agenda.

Stage 3: Newsroom Review & Resource Allocation

- **Detailed Asset Audit:** The editorial team conducts an in-depth curation check in an Editorial Meeting, verifying whether the story has necessary supporting assets such as text, audio, and photography.
- **Resolving Asset Deficits:** If a story lacks key visuals or audio coverage, the team takes one of two approaches:
    - **Option A (Internal Archive):** Source existing media from the platform's central archive.
    - **Option B (Field Reporting - Preferred):** Commission a local reporters with the relevant badges, in the relevant region to provide on-the-ground coverage (audio, photography, or reporting).

Stage 4: Drafting & Story Assembly

- **Resource Verification:** Once all requested multimedia assets and reporting elements are delivered, the newsroom confirms that all associated requirements are satisfied[1].
- **Final Copywriting:** A staff writer synthesizes the reporting and materials to produce the final written piece.

Stage 5: Editor-in-Chief (Director/Coordinator) Review & Website Publication

- **Executive Quality Control:** The Editor-in-Chief conducts the final review, ensuring proper grammar, data accuracy, structure, and adherence to editorial standards.
- **Single-Article Web Release:** Upon final sign-off, the Editor-in-Chief publishes the individual article directly to the main website.

Stage 6: Newsletter Layout & Compilation

- **Curating Published Stories:** A graphic/layout designer gathers the stand-alone articles previously published on the website to build the weekly newsletter.
- **Editorial & Space Adjustments:** When layout or length constraints prevent including all pieces, lower-ranked articles (e.g., 3-star vs. 5-star stories) may be cut or adjusted, requiring direct consultation and approval from the Editor-in-Chief.

Stage 7: Revision & Feedback Loop

- **Rejection / Rework Protocol:** If the Editor-in-Chief or editorial team identifies missing information, incomplete assets, or unmet quality benchmarks at any checkpoint, the article is sent back to previous stages for revision before publication.

Initially, the concept described above should be able to be sustained with the following infrastructure:

Technical Layers:

1. **The Public Newspaper Website**
    - **Concept:** The highly cached, read-optimized front-end interface where the public consumes the media and articles.

   Cost-Effective Option: Build a statically generated site using Next.js or Hugo, and host it on Cloudflare Pages or GitHub Pages. Like printing the newspaper in advance and handing out copies. It is fast, secure, and can handle a million readers at once without crashing or costing extra.

2. **The Newsroom Application**
    - **Concept:** The internal dashboard and workspace. Where editors collaborate, review AI extractions, and manage the publishing calendar. The "Staff-Only Back Office." The digital desks where reporters type drafts and editors make corrections.

   Cost-Effective Option: Use an open-source headless Content Management System like Strapi or Ghost. (Probably Strapi) They can be hosted on Azure with non-profit credit. Alternatively, if the team is small, Notion often provides non-profit discounts and can serve as a highly flexible, database-driven editorial workspace.

3. **The Back End API**
    - **Concept:** The central nervous system. It routes traffic, enforces business logic, triggers AI pipelines, and bridges the newsroom with the public site.

   Cost-Effective Option: Python (FastAPI) or Node.js (Express) deployed as serverless functions (Azure Functions) or on a Azure App Service instance.

4. **The Database**
    - **Concept:** The structured relational core. It holds user profiles, metadata, story states, publishing dates, and system logs keeping lightweight text data separate from heavy media.

   Cost-Effective Option: PostgreSQL is the industry standard. You can host a managed PostgreSQL instance directly on Azure. Or try MariaDB for free. But it Will require a lot of maintenance

5. **The File Archive**
    - **Concept:** Your digital asset management (DAM) system for raw, heavy media. Audio recordings, raw photography, and uncompressed video live here.

    Cost-Effective Option: The drive Huella already has

6. **Google Drive Integration**
    - **Concept:** The programmatic bridge. It allows the Back End API to read, write, and organize files in the Drive archive without human intervention.

    Cost-Effective Option: Google Cloud Console allows to generate a Service Account. Calling the Google Drive API via a service account to shuffle files to and from your 100 TB archive is allegedly free.

7. **The Submission System**
    - **Concept:** The intake funnel for reporters. It requires file upload capabilities and must securely pipe data into the API and Archive.

    Cost-Effective Option: For general intakes, Tally.so has a robust free tier, or we can try to build a custom React form.

8. **The AI Processing Layer**
    - **Concept:** The raw intelligence engines that execute specific tasks: Whisper for audio transcription, layout-aware models for document OCR, and LLMs for summarizing or tagging entities.

    Cost-Effective Option: Run open-source models like Llama 3 or Whisper locally on Azure VMs. For managed APIs, Groq or Gemini might be worth looking into.

9. **The AI Orchestration Layer**
    - **Concept:** The cognitive supervisor. It decides *when* to call the processing layer, chains prompts together, handles tool execution, and manages context windows

    Cost-Effective Option: LangChain, LlamaIndex, or CrewAI are entirely open-source and free Python frameworks. Might be worth looking i'm tu o a custom orchestrator in Python, gives you the most control and costs nothing but developer time. Here is where I could play around with Odysseus

10. **The Rules Engine**
    - **Concept:** The deterministic filter. It applies rigid organizational criteria—like flagging local keywords, evaluating urgency, or prioritizing based on predefined editorial rules—before human review.

Cost-Effective Option: Probably the Best is to write our own

11. **Search and Archive Discovery**
    - **Concept:** The retrieval system for journalists to query past articles, notes, and metadata, transitioning from basic keyword matching to vector-based semantic search ("find me stories about water pollution").

Cost-Effective Option: Probably Best to work with PostgreSQL's. But maybe worth keeping out of the MVP.

12. **Authentication and Permissions**
    - **Concept:** Identity and access management (IAM).

Cost-Effective Option: Clerk or Auth0 have solid free tiers for startups and small organizations. Alternatively, Microsoft Entra ID (formerly Azure AD), fits with Azure's suite. Other than that, we might have to hard code most things to fit our needs. Is the most cost effective, and most secure option. ID Checks are a whole other can of worms, can be left out from the MVP.

13. **Notifications and Communication**
    - **Concept:** The alerting system. It pings editors when a new submission arrives, updates reporters on story states, and sends newsletters to readers.

Cost-Effective Option: SendGrid and Resend offer free tiers for high-deliverability transactional emails. Discord is entirely free for internal webhooks and team alerts, while Slack offers non-profit discounts. It sounds quite simple to make Google apps scripts to send emails to a table of Users.

14. **Infrastructure and Operations**
    - **Concept:** The DevOps backbone. Where the code lives, how it deploys, and how you ensure the systems stay online and secure.

Cost-Effective Option: GitHub offers discounted or free Enterprise features for non-profits. Use GitHub Actions (free tier) for CI/CD pipelines to deploy code to Azure.

15. The Editorial Workflow State Machine

- **Concept:** The rigid pipeline tracker that prevents a story from moving to "Published" without passing through "Transcribed," "Draft," and "Editor Review."

Cost-Effective Option: This does not require separate software. Manage state transitions using simple Enums and timestamp checks on PostgreSQL database. There are also open-source libraries like XState that can help with more complex frontend state visualization.